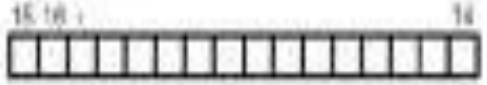
LCD 16x2 Interfacing

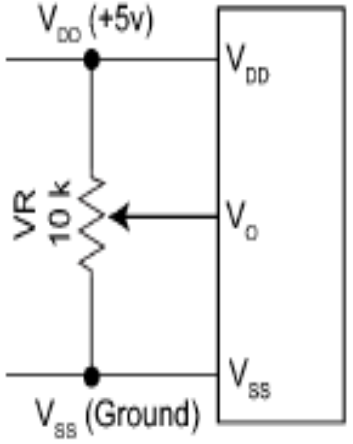
LCD PINOUTS

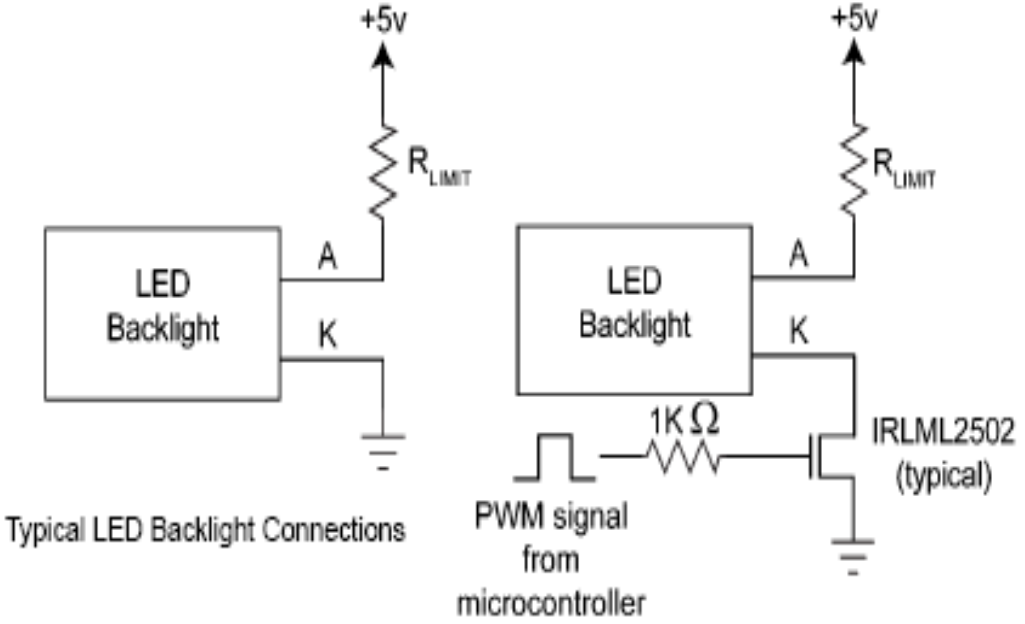
LCD: LCD1



Pin Number	Signal Name
1	GND
2	+5V
3	LCD Contrast
4	RS
5	R/W
6	E
7 – 14	D0 – D7
15	Backlight +
16	Backlight -

Connector LCD1 Pin Out





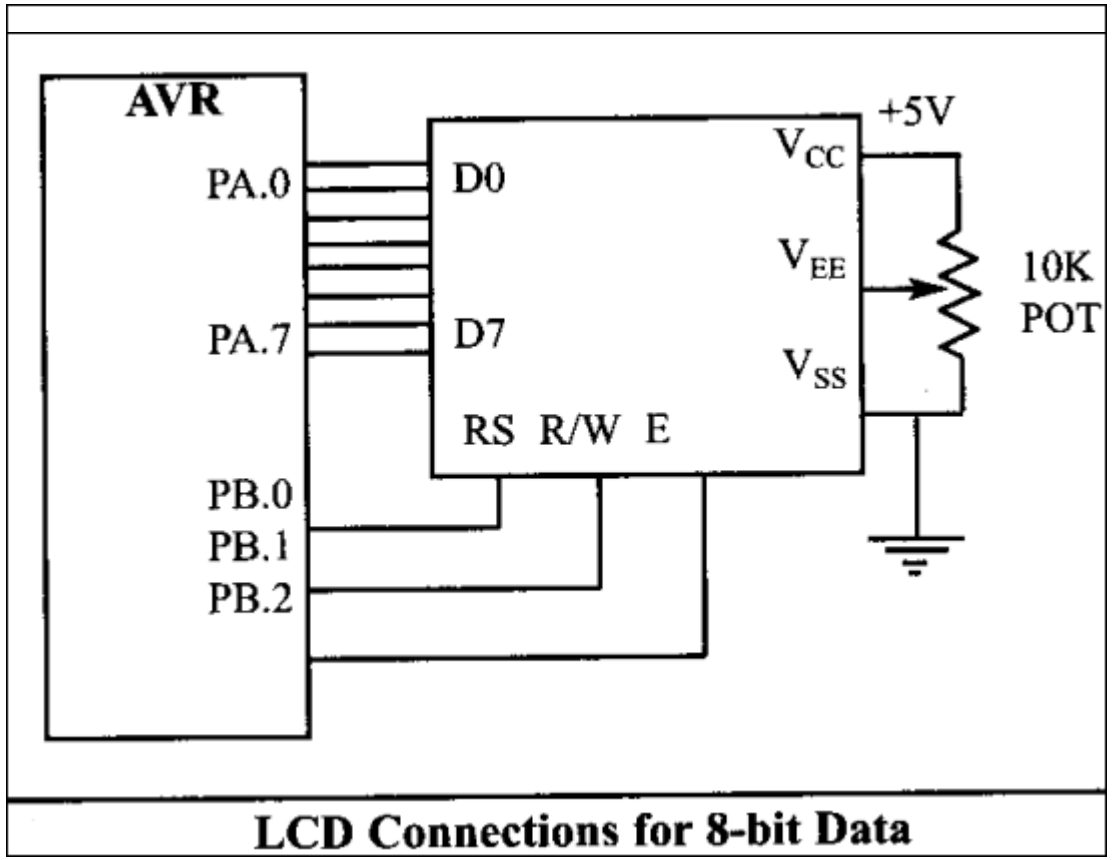
The backlight in this LCD is composed of LEDs in series. The total voltage drop across these LEDs is typically 4.2 V and the recommended current through the LEDs is 120 mA. You should use a current limiting resistor $\mathbf{R_{LIMIT}}$ as shown above where:

$$\mathbf{R_{LIMIT} = (V_{BACKLIGHT} - 4.2V) / 0.12A}$$

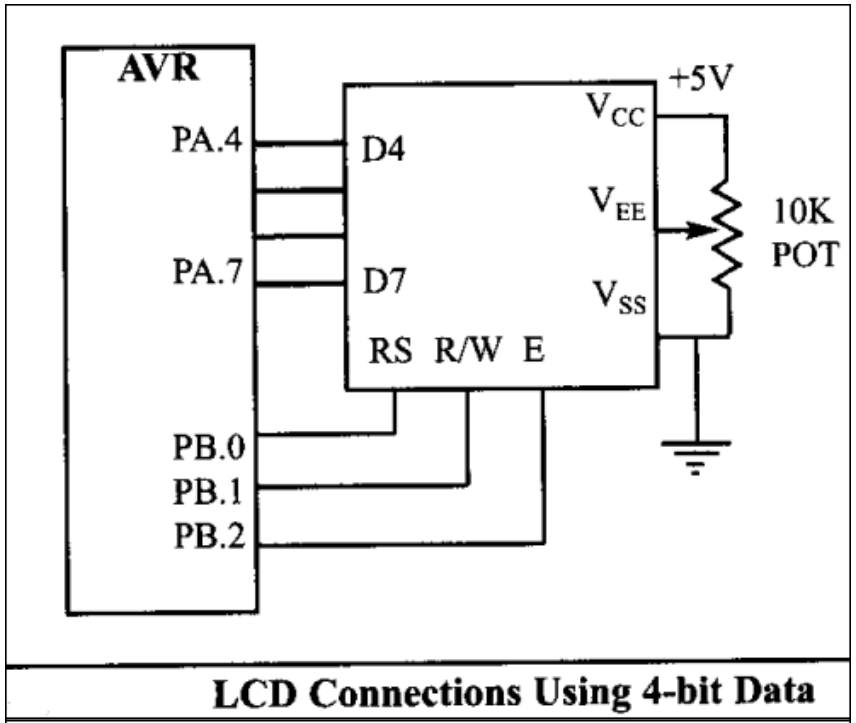
PIN Description for LCD

Pin(s)	Symbol	I/O	Description
1	V_{SS}	--	Ground
2	V_{cc}	--	+5V power supply
3	V_{EE}	--	Power supply to control contrast
4	RS	I	RS=0 to select command register RS=1 to select Data register
5	R/W	I	R/W=0 for write R/W=1 for read
6	E	I/O	Enable (H to L pulse for 450ns)
7 - 14	DBx	I/O	8-bit data bus (x = 7 -14)

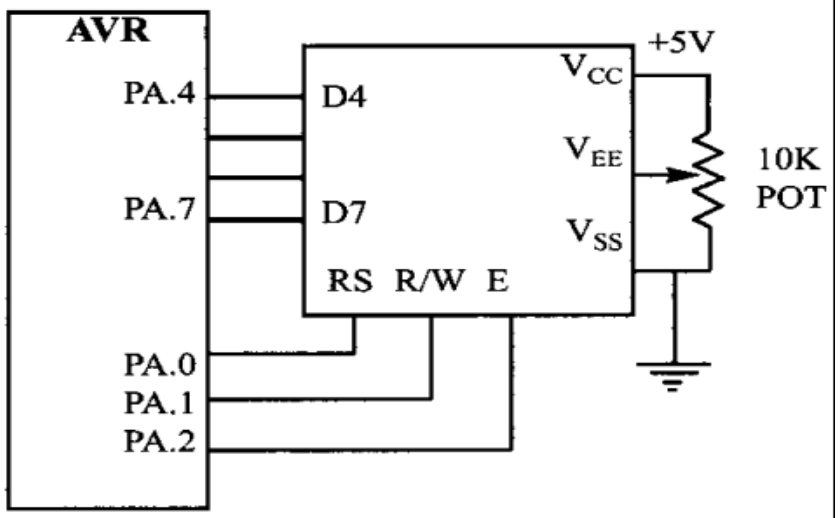
RS = 1 (Data Register)	RS = 0 (Command Register)	
ASCII Codes (A-Z, a-z, 0-9)	1	Clear display screen
	2	Return home
	4	Decrement cursor (shift cursor to left)
	6	Increment cursor (shift cursor to right)
	5	Shift display right
	7	Shift display left
	8	Display off, cursor off
	A	Display off, cursor on
	C	Display on, cursor off
	E	Display on, cursor blinking
	F	Display on, cursor blinking
	10	Shift cursor position to left
	14	Shift cursor position to right
	18	Shift the entire display to the left
	1C	Shift the entire display to the right
	80	Force cursor to beginning of 1st line
	C0	Force cursor to beginning of 2nd line
	28	2 lines and 5 × 7 matrix (D4–D7, 4-bit)
	38	2 lines and 5 × 7 matrix (D0–D7, 8-bit)



LCD Connections for 8-bit Data



LCD Connections Using 4-bit Data



8-bit MODE
Two PORTs

Initialize the LCD

Required	Command
5 x 7 matrix	0x38
LCD Mode of operation (Display on, Cursor Blinking)	0x0E
Clear Display Screen	0x01 (0x01 and 0x02 wait for 2ms)

Sending commands to the LCD

Make pins RS = 0 and R/W = 0
Put command value on data pins (D0-D7)
Send H-to-L pulse to E pin to latch data on LCD internal register
After each command should wait for 100µs

Sending Data to the LCD

Make pins RS = 1 and R/W = 0
Put data on pins (D0-D7)
Send H-to-L pulse to E pin to latch data on LCD internal register
After each command should wait for 100µs

```
#include<avr/io.h>
#include<util/delay.h>
#define LCD_DPRT PORTK  //LCD DATA PORT
#define LCD_DDDR DDRK  //LCD DATA DDR
#define LCD_DPIN PINK  //LCD DATA PIN
#define LCD_CPRT PORTF  //LCD COMMANDS PORT
#define LCD_CDDR DDRF  //LCD COMMANDS DDR
#define LCD_CPIN PINF  //LCD COMMANDS PIN
#define LCD_RS  0      //LCD RS
#define LCD_RW  1      //LCD RW
#define LCD_EN  2      //LCD EN
//*****************************************************************
void lcdcommand(unsigned char cmnd)
{
  LCD_DPRT = cmnd;          //send cmnd to data port
  LCD_CPRT &= ~(1<<LCD_RS); //RS = 0 for command
  LCD_CPRT &= ~(1<<LCD_RW); //RW = 0 for write
  LCD_CPRT |= (1<<LCD_EN);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  LCD_CPRT &= ~(1<<LCD_EN); //EN = 1 for H-to-L pulse
  _delay_us(100);           //wait tomakeenable wide
}
```

```
//*****************************************************************
void lcddata(unsigned char data)
{
  LCD_DPRT = data;          //send data to data port
  LCD_CPRT |= (1<<LCD_RS);  //RS = 1 for data
  LCD_CPRT &= ~(1<<LCD_RW); //RW = 0 for write
  LCD_CPRT |= (1<<LCD_EN);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  LCD_CPRT &= ~(1<<LCD_EN); //EN = 1 for H-to-L pulse
  _delay_us(100);           //wait tomakeenable wide
}
//*****************************************************************
void lcd_init()
{
  LCD_DDDR = 0xFF;
  LCD_CDDR = 0xFF;

  LCD_CPRT &= ~(1<<LCD_EN); //LCD_EN = 0
  _delay_us(2000);          //wait for initilization
  lcdcommand(0x38);         //init LCD 2 line, 5x7 matrix
  lcdcommand(0x0E);         //display ON, Cursor Blinking
  lcdcommand(0x01);         //clear LCD
```

```
 _delay_us(2000);          //wait
  lcdcommand(0x06);         //shift cursor right
}
//*****************************************************************
int main(void)
{
  lcd_init();
  lcdcommand(0x80);
  lcddata('A');
  while(1);
  return 0;
}
```

4-bit MODE
Two PORTs

Initialize the LCD

Required	Command
For Initialization of 4-bit mode	0x33
For Initialization of 4-bit mode	0x32
5 x 7 matrix	0x28
LCD Mode of operation (Display on, Cursor Blinking)	0x0E
Clear Display Screen	0x01 (0x01 and 0x02 wait for 2ms)

Sending commands to the LCD

Make pins RS = 0 and R/W = 0
Put command value on data pins (D4-D7)
Send H-to-L pulse to E pin to latch data on LCD internal register
After each command should wait for 100µs

Sending Data to the LCD

Make pins RS = 1 and R/W = 0
Put data on pins (D4-D7)
Send H-to-L pulse to E pin to latch data on LCD internal register
After each command should wait for 100µs

```
#include<avr/io.h>
#include<util/delay.h>
#define LCD_DPRT PORTK  //LCD DATA PORT
#define LCD_DDDR DDRK  //LCD DATA DDR
#define LCD_DPIN PINK  //LCD DATA PIN
#define LCD_CPRT PORTF  //LCD COMMANDS PORT
#define LCD_CDDR DDRF  //LCD COMMANDS DDR
#define LCD_CPIN PINF  //LCD COMMANDS PIN
#define LCD_RS  0      //LCD RS
#define LCD_RW  1      //LCD RW
#define LCD_EN  2      //LCD EN
//*****************************************************************
void lcdcommand(unsigned char cmnd)
{
  LCD_DPRT = cmnd & 0xF0;   //send high nibble to D4-D7
  LCD_CPRT &= ~(1<<LCD_RS); //RS = 0 for command
  LCD_CPRT &= ~(1<<LCD_RW); //RW = 0 for write
  LCD_CPRT |= (1<<LCD_EN);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  LCD_CPRT &= ~(1<<LCD_EN); //EN = 1 for H-to-L pulse
  _delay_us(100);           //wait
  LCD_DPRT = cmnd << 4;   //send Low nibble to D4-D7
```

```
 LCD_CPRT |= (1<<LCD_EN);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  LCD_CPRT &= ~(1<<LCD_EN); //EN = 1 for H-to-L pulse
  _delay_us(100);          //wait
}
//*****************************************************************
void lcddata(unsigned char data)
{
  LCD_DPRT = data & 0xF0;   //send high nibble to D4-D7
  LCD_CPRT |= (1<<LCD_RS);  //RS = 1 for data
  LCD_CPRT &= ~(1<<LCD_RW); //RW = 0 for write
  LCD_CPRT |= (1<<LCD_EN);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  LCD_CPRT &= ~(1<<LCD_EN); //EN = 1 for H-to-L pulse
  _delay_us(100);          //wait
  LCD_DPRT = data << 4;   //send Low nibble to D4-D7
  LCD_CPRT |= (1<<LCD_EN);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  LCD_CPRT &= ~(1<<LCD_EN); //EN = 1 for H-to-L pulse
  _delay_us(100);          //wait
}
```

```
//*****************************************************************
void lcd_init()
{
  LCD_DDDR = 0xFF;
  LCD_CDDR = 0xFF;

  LCD_CPRT &= ~(1<<LCD_EN); //LCD_EN = 0
  _delay_us(2000);          //wait for initilization
  lcdcommand(0x33);
  lcdcommand(0x32);
  lcdcommand(0x28);         //init LCD 2 line, 5x7 matrix
  lcdcommand(0x0E);         //display ON, Cursor Blinking
  lcdcommand(0x01);         //clear LCD
  _delay_us(2000);          //wait
  lcdcommand(0x06);         //shift cursor right
}
//*****************************************************************
int main(void)
{
  lcd_init();
  lcdcommand(0x80);
  lcddata('I');
```

4-bit MODE
Single PORT

```
#include<avr/io.h>
#include<util/delay.h>
void lcdcommand(unsigned char cmnd)
{
  PORTK = cmnd;          //send cmnd to data port
  PORTF &= ~(1<<0); //RS = 0 for command
  PORTF &= ~(1<<1); //RW = 0 for write



  PORTF  |= (1<<2);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  PORTF &= ~(1<<2); //EN = 1 for H-to-L pulse
  _delay_us(100);          //wait tomakeenable wide
}
void lcddata(unsigned char data)
{
 PORTK= data;          //send data to data port
 PORTF |= (1<<0);  //RS = 1 for data
PORTF &= ~(1<<1); //RW = 0 for write
 PORTF  |= (1<<2);  //EN = 1 for H-to-L pulse
  _delay_us(1);
```

```
void lcd_init()
{
  DDRK = 0xFF;
  DDRF = 0xFF;
   PORTF &= ~(1<<2); //LCD_EN = 0
  _delay_us(2000);          //wait for initilization
  lcdcommand(0x38);         //init LCD 2 line, 5x7 matrix



  lcdcommand(0x0E);         //display ON, Cursor Blinking
  lcdcommand(0x01);         //clear LCD
 _delay_us(2000);          //wait
  lcdcommand(0x06);         //shift cursor right
}
//*****************************************************************
int main(void)
{
  lcd_init();
  lcdcommand(0x80);
  lcddata('A');
  while(1);
  return 0;
```

RS = 1 (Data Register)	RS = 0 (Command Register)	
ASCII Codes (A-Z, a-z, 0-9)	1	Clear display screen
	2	Return home
	4	Decrement cursor (shift cursor to left)
	6	Increment cursor (shift cursor to right)
	5	Shift display right
	7	Shift display left
	8	Display off, cursor off
	A	Display off, cursor on
	C	Display on, cursor off
	E	Display on, cursor blinking
	F	Display on, cursor blinking
	10	Shift cursor position to left
	14	Shift cursor position to right
	18	Shift the entire display to the left
	1C	Shift the entire display to the right
	80	Force cursor to beginning of 1st line
	C0	Force cursor to beginning of 2nd line
	28	2 lines and 5 × 7 matrix (D4–D7, 4-bit)
	38	2 lines and 5 × 7 matrix (D0–D7, 8-bit)

```
void lcdcommand(unsigned char cmnd)
{
  PORTK &= ~(1<<0); //RS = 0 for command
  PORTK &= ~(1<<1); //RW = 0 for write
  PORTF = cmnd;          //send cmnd to data port
  PORTK |= (1<<2);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  PORTK &= ~(1<<2); //EN = 1 for H-to-L pulse
  _delay_us(100);          //wait tomakeenable wide
}
//*****************************************************************
void lcddata(unsigned char data)
{
  PORTK |= (1<<0);  //RS = 1 for data
  PORTK &= ~(1<<1); //RW = 0 for write
  PORTF = data;          //send data to data port
  PORTK |= (1<<2);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  PORTK &= ~(1<<2); //EN = 1 for H-to-L pulse
  _delay_us(100);          //wait tomakeenable wide
}
```

```
//*****************************************************************
void lcd_init()
{
  DDRK = 0xFF;
  DDRF = 0xFF;
  PORTK &= ~(1<<2); //LCD_EN = 0
  _delay_us(2000);          //wait for initilization
  lcdcommand(0x38);         //init LCD 2 line, 5x7 matrix
  lcdcommand(0x0E);         //display ON, Cursor Blinking
  lcdcommand(0x01);         //clear LCD
 _delay_us(2000);          //wait
  lcdcommand(0x06);         //shift cursor right
}
//*****************************************************************
int main(void)
{
  lcd_init();
  lcdcommand(0x80);
  lcddata('A');
  while(1);
  return 0;
}
```

```
#include<avr/io.h>
#include<util/delay.h>
unsigned char received;
void lcdcommand(unsigned char cmnd)
{
  PORTF &= ~(1<<0); //RS = 0 for command
  PORTK = cmnd;         //send cmnd to data port
  PORTF |= (1<<1);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  PORTF &= ~(1<<1); //EN = 1 for H-to-L pulse
  _delay_us(100);          //wait tomakeenable wide
}
//*************************************************************
void lcddata(unsigned char data)
{
  PORTF |= (1<<0);  //RS = 1 for data
  PORTK = data;     //send data to data port
  PORTF |= (1<<1);  //EN = 1 for H-to-L pulse
  _delay_us(1);
  PORTF &= ~(1<<1); //EN = 1 for H-to-L pulse
  _delay_us(100);          //wait tomakeenable wide
}
//*************************************************************
void lcd_init()
```

```
#include<avr/io.h>
#include<util/delay.h>

void lcdcomnd(unsigned char cmnd)
{
 PORTB &= ~(1<<0);
 PORTB &= ~(1<<1);
 PORTF = cmnd;
 PORTB |= (1<<2);
 _delay_us(1);
 PORTB &= ~(1<<2);
 _delay_us(100);
}
void lcddata(unsigned char data)
{
 PORTB |= (1<<0);
 PORTB &= ~(1<<1);
 PORTF = data;
 PORTB |= (1<<2);
 _delay_us(1);
 PORTB &= ~(1<<2);
 _delay_us(100);
}
void lcd_init()
{
```